Governed Multi-Agent GraphRAG for Banking Regulatory Intelligence

Phase 2 (AWS V1)

From GraphRAG Proof of Concept to a Production-Oriented, Governed LangGraph-Based Multi-Agent Architecture on AWS

LangGraph | Microsoft GraphRAG | OpenAI GPT-4.1 | DeepSeek | AWS ECS

Jingru Chen • Phase 2 Complete — September 7, 2026

From Phase 1 Foundation to Phase 2 Delivered Architecture

The portfolio evolves from evidence-grounded GraphRAG retrieval into a governed, deployable multi-agent AI runtime.

PHASE 1 — FOUNDATION

Microsoft GraphRAG PoC

- 23-document banking / regulatory corpus
- Entity, relationship & community indexing
- Basic / Local / Global Search
- Evidence validation & citation checks
- Runtime guardrails
- Public Streamlit demonstration

ENGINEERING GAP

What Phase 1 exposed

- Retrieval quality can be uneven
- Citation presence ≠ claim support
- Exact identifiers / aliases need resolution
- Provider behavior differs
- Long-running Global Search needs production controls
- Recovery and escalation must be explicit

PHASE 2 (AWS V1) — DELIVERED

Governed Multi-Agent GraphRAG

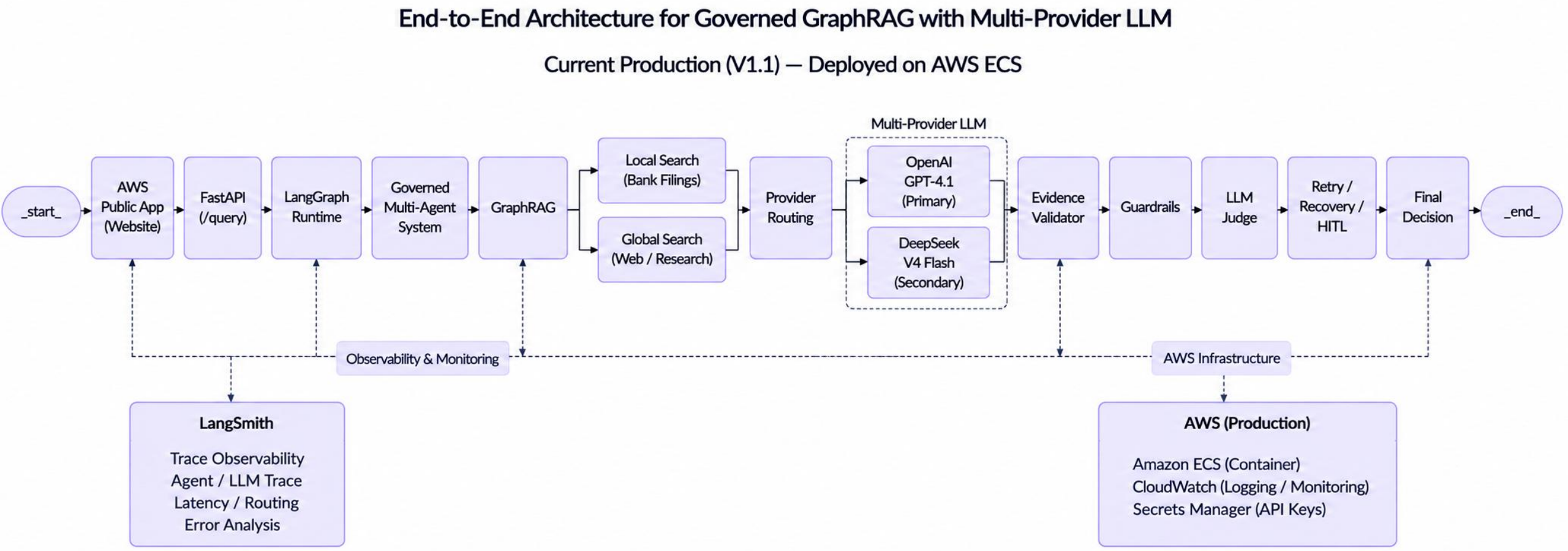
- LangGraph orchestration
- Planning / research / validation workflow
- Deterministic guardrails + LLM judge
- Retry / recovery / escalation / human review
- OpenAI primary + DeepSeek secondary architecture
- Docker → ECR → ECS public AWS deployment

Phase1_Live_Memo

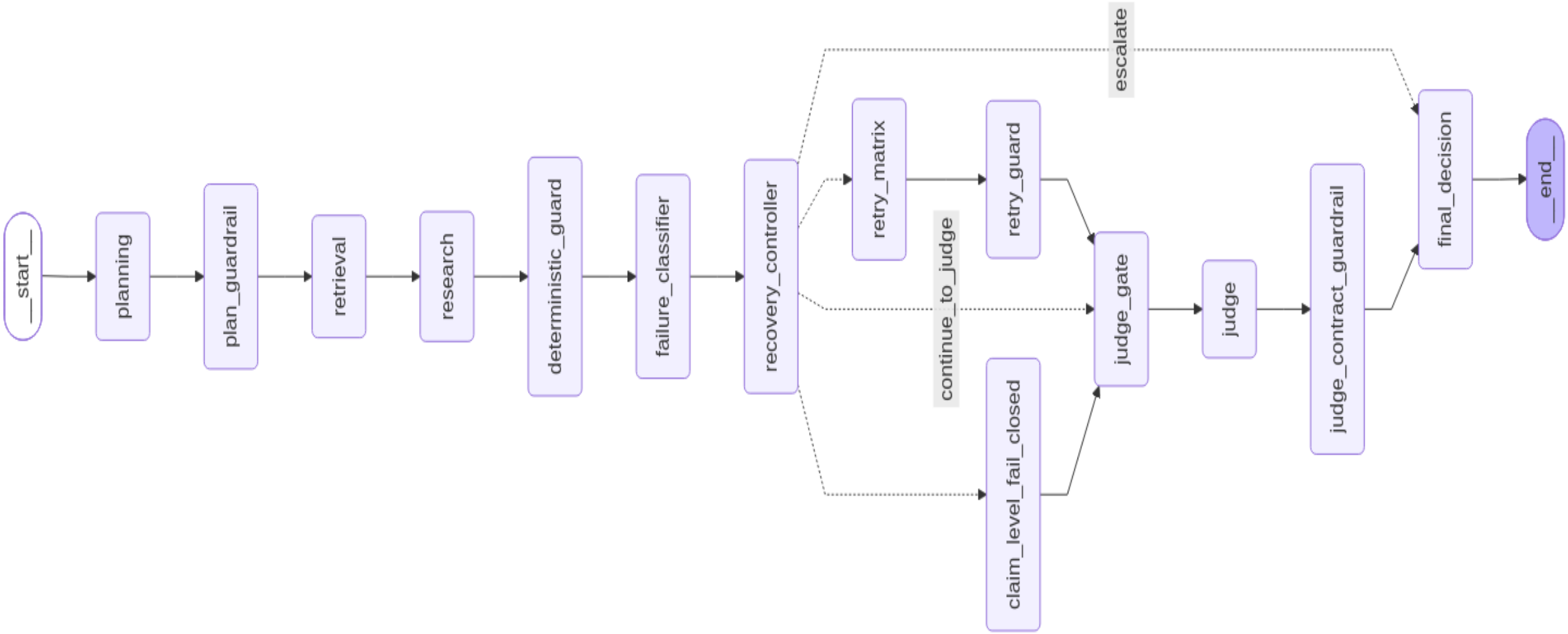
Phase2_AWS_V1_Live_Memo

Engineering story: identify failure modes → design governed orchestration → validate end-to-end → deploy on AWS.

AWS V1 Process Workflow



LangGraph Governed Multi-Agent Workflow



Governance, Recovery & Human Review

Defense-in-depth controls turn retrieval quality, provider behavior, and evidence sufficiency into observable runtime decisions.

1 PREVENT

Plan Guardrail

- Validate request / plan
- Bound unsupported behavior
- Preserve governed routing

2 DETECT

Deterministic Guard

- Evidence sufficiency
- Citation / provenance checks
- Missing-evidence detection
- Failure classification

3 RECOVER

Recovery Controller

- Continue to judge
- Retry matrix
- Retry guard
- Claim-level fail-closed

4 JUDGE

Independent LLM Judge

- Separate evaluation step
- Judge contract guardrail
- Approve or reject governed output

5 ESCALATE

Human Review

- PENDING_REVIEW is a valid governed outcome
- Escalate unresolved risk instead of hallucinating

Governance objective: make failure visible, recoverable, auditable, and safe—not merely maximize answer rate.

Layered Guardrail System — Defense in Depth

Guardrails operate across planning, retrieval, evidence, output contracts, recovery, and independent review—not as a single post-processing check.

L1 REQUEST & PLAN

Prevent invalid or unsafe execution

- Validate request and plan structure
- Bound unsupported / out-of-scope behavior
- Preserve selected provider and governed routing
- Reject malformed planning outputs

L2 RETRIEVAL & SCOPE

Constrain what evidence may enter

- Bank / document / period scope checks
- Canonical source-ID handling
- Exact identifier / alias resolution
- Trusted-source and span-catalog boundaries

L3 EVIDENCE & PROVENANCE

Prove every factual claim

- Evidence sufficiency / missing-evidence checks
- Citation-to-source provenance validation
- Exact quote / span validation
- Value, unit, period & scope field evidence

L4 OUTPUT CONTRACT

Validate the research matrix

- Single-bank row / schema contract
- ≤ 4 unique evidence spans per row
- Supported ⇒ evidence for every required field
- No validated governed output ⇒ no public answer

L5 FAILURE & RECOVERY

Fail closed, then recover selectively

- Failure classification
- Continue-to-Judge only after Guard PASS
- Matrix-only retry / retry guard
- Claim-level fail-closed / escalation

L6 JUDGE & HUMAN REVIEW

Independent final governance

- Independent LLM Judge review
- Judge-contract guardrail
- APPROVED vs PENDING_REVIEW decision
- Escalate unresolved risk to human review

Control principle: validate early → ground every claim → recover narrowly → judge independently → escalate when uncertainty remains.

AWS Deployment & End-to-End Validation

V1 was frozen only after container deployment, ECS rollout stability, health checks, and governed Local Search validation.

DEPLOYMENT PATH

Application + GraphRAG runtime
↓
Docker container
↓
Amazon ECR
↓
Amazon ECS public service

AWS V1 FREEZE EVIDENCE

ECS rollout: COMPLETED
Desired: 1 Running: 1
Failed: 0 Pending: 0

Health endpoint: healthy
Orchestrator: LangGraph
Migration status: CLOSED_PASS

LOCAL SEARCH E2E

AWS Local Search: PASS

Example validated case:
Citigroup 2025 Q3 provision = $2,259M
Guardrail: PASS
Judge: PASS
Final decision: APPROVED

GLOBAL SEARCH — IMPORTANT PRODUCTION FINDING

GraphRAG Global Search was validated in the application runtime (~9.4 min), but the public AWS synchronous path hit a 60-second ingress timeout (HTTP 504). V1 therefore exposes Global Search as EXPERIMENTAL rather than claiming synchronous AWS E2E success.
V2 architecture: start job → return job_id → background execution → status polling / result retrieval.

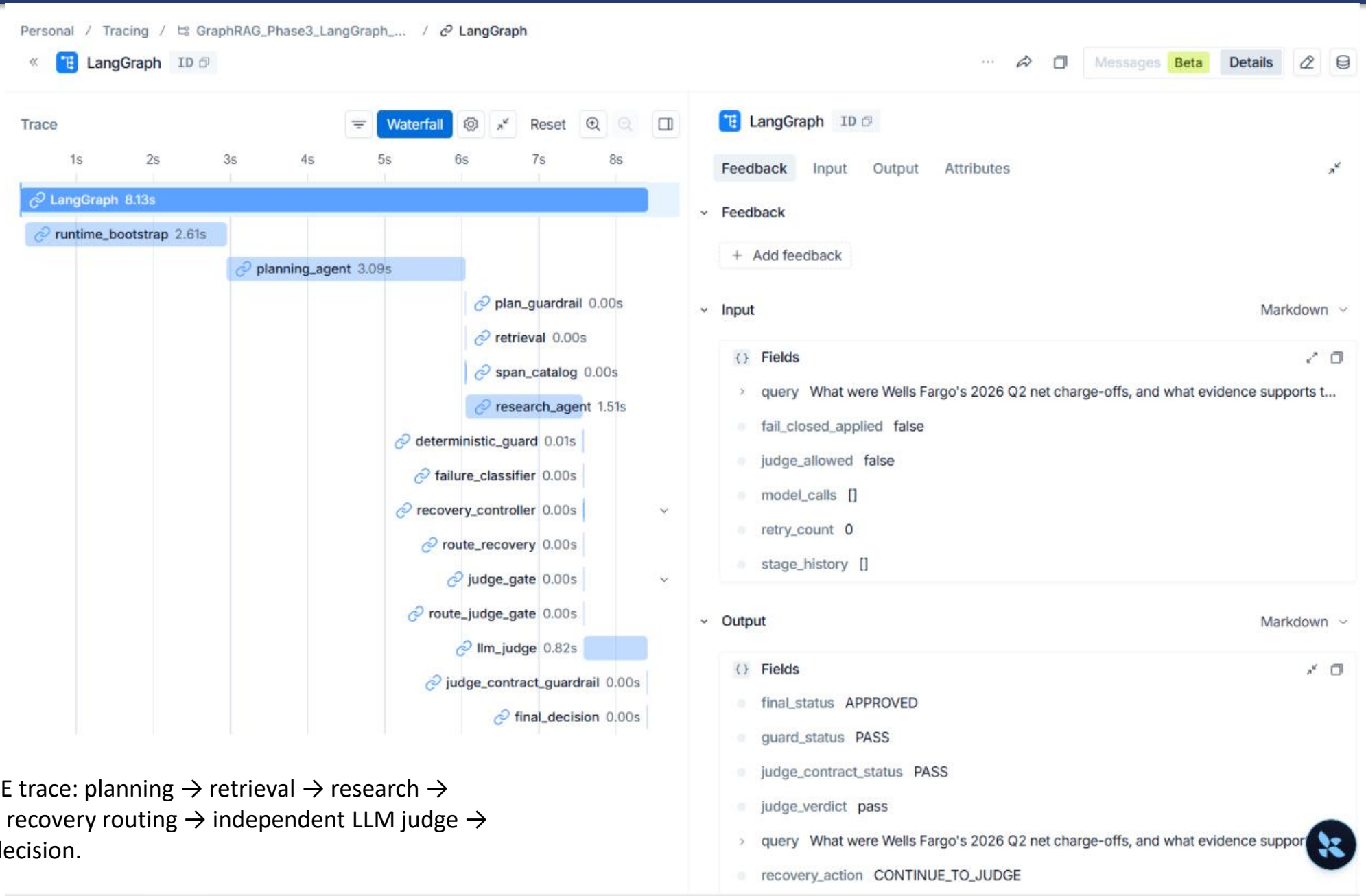
LOCAL: AWS E2E PASS

GLOBAL BACKEND: VALIDATED

AWS SYNC: 60s 504

V1: EXPERIMENTAL

E2E Runtime Validation & Observability (LangSmith w/ Local Search)



Observed E2E trace: planning → retrieval → research → guardrails → recovery routing → independent LLM judge → APPROVED decision.